Review Lösung Julius P. (Aufgabe 4-2)

F1 - Wiederverwendung & Singleton

- Wiederverwendung: Kopieren statt Generics ist eine unglückliche Lösung, da bei einer Kopie auch Fehler mit kopiert werden, die dann in allen Kopien gefixed werden müssen. Besser wäre die Verwendung von Generics mit `<E extends Member>` (oder wenn man statt Member das Interface in "Identifiable" umbenennen würde, wäre treffender). Hierbei zu beachten: Generics und Singleton beißen sich, weil eine statische Instanz keinen Typparameter haben kann. Lösung über eine typspezifische Unterklasse oder Dependency Injection.
- Singleton: Umsetzung nicht gut. Konstruktor ist package-private nicht private, also alles im Package kann den Konstruktor aufrufen. Das fehlende final bei instance sorgt dafür, dass es doch mehrere Instanzen geben kann, weil mit instance = new Container() die alte Instanz überschrieben werden kann.
- Thread-Safety / Speicher: Die Instanz wird direkt beim Laden der Klasse durch die JVM erzeugt (eager initialization). Die JVM garantiert, dass das nur einmal passiert, daher ist die Variante thread-safe. Nachteil: Die Instanz wird auch dann erzeugt und belegt Speicher, wenn sie nie gebraucht wird. Gegenvariante ist die lazy initialization, bei der `getInstance()` prüft, ob es schon eine Instanz gibt, und sie erst dann erzeugt. Diese ist ohne zusätzliche Synchronisation (z. B. `synchronized`) allerdings nicht thread-safe.

F2 - Alles in einer Klasse

- Die Behauptung stimmt nicht. `Container` übernimmt Listenverwaltung, Speichern, Laden, Eingabe, Ausgabe und den Programmstart gleichzeitig. Dadurch wird die Klasse unübersichtlich und schwer wartbar.
- Jede Änderung (z. B. GUI statt Konsole oder Datenbank statt Datei) betrifft dieselbe Klasse und kann ungewollt andere Funktionen beeinflussen.
- Damit wird das Single Responsibility Principle (das „S“ in SOLID) verletzt: Jede Klasse sollte nur eine Zuständigkeit und damit nur einen Grund zur Änderung haben.

F3 - store()-Exception und Scanner

- Exception bei `store()`: Julius' `UserStory` implementiert nicht `Serializable`. Deshalb wirft `writeObject()` eine `NotSerializableException` (eine `IOException`), die in eine `ContainerException` umgewandelt wird.
- Programmende: Weder `startEingabe()` noch `main` fangen die Exception ab. Beide reichen sie nur per `throws` weiter, sodass sie bis zur JVM durchläuft und das Programm abbricht. Besser: die Exception in der Eingabeschleife abfangen, eine Fehlermeldung ausgeben und weiterlaufen.
- Begrüßung: Das `println` mit der Begrüßung steht innerhalb der `while`-Schleife und wird deshalb bei jedem Durchlauf erneut ausgegeben. Es gehört einmalig vor die Schleife.
- Scanner: `nextLine()` liest die gesamte Eingabezeile bis zum Enter und gibt sie als String zurück. Diese wird anschließend per `split` in Befehl und Parameter zerlegt.

F4 - Sortierung & Attribute UserStory

- Sortierung: Ein eigener Sortieralgorithmus ist unnötig. Aus dem JDK kann z. B. ein `Comparator` wiederverwendet werden (`Comparator.comparingDouble(...)` mit `reversed()`, zusammen mit `List.sort` oder `stream().sorted(...)`). Sortiert werden sollte absteigend nach der Priorität.
- Attribute unvollständig: Es fehlen Mehrwert, Strafe, Aufwand und Risiko sowie ein Akzeptanzkriterium.
- Die Priorität wird von außen gesetzt (`setPrio`) statt aus den Werten berechnet. Dadurch kann sie beliebige, inkonsistente Werte annehmen.

- Sichtbarkeiten: `id`, `prio` und `project` sind package-private statt `private`. Sie können also von jeder Klasse im Package direkt verändert werden, und die Kapselung ist verletzt.

F5 - Filter-Map-Reduce

- Julius verwendet für die Ausgabe keine Streams, sondern nur eine klassische for-each-Schleife. Die Filterung fehlt komplett.
- Mit Streams lässt sich das kompakt lösen: `filter()` mit einer Lambda-Expression, in der mehrere Bedingungen mit `&&` verknüpft werden (z. B. Projekt = „Coll@HBRS“ und Prio ≥ 3), anschließend `sorted()` mit einem `Comparator` und dann die Ausgabe.
- Es müssen nicht alle drei Schritte zum Einsatz kommen. In der eigenen Lösung wurden nur Filter und Sortierung benötigt, kein Map und kein Reduce.
- Ein Blick in die Musterlösung der Aufgabe 2-2 hätte Julius hier vermutlich weitergeholfen.

F6 - Package-Struktur & Muster

- Sinnvolle Zerlegung nach Zuständigkeiten, z. B.:
  - `entities`: Fachobjekte (`UserStory`)
  - `control`: Verwaltung der Objekte (`Container`)
  - `persistence`: Speichern und Laden
  - `view`: Ein- und Ausgabe über die Konsole
- Als Muster dient hier Model-View-Controller (MVC): Model (`entities`), View (`view`) und Controller (`control`) werden voneinander getrennt.
- Für die Persistenz eignet sich zusätzlich das Strategy-Pattern, damit die Speicherart (z. B. Datei oder Datenbank) austauschbar ist, ohne den Container zu ändern.